МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

КУРСОВАЯ РАБОТА

по дисциплине «Алгоритмы и структуры данных»

Тема: Выбор и реализация наиболее подходящей структуры данных

Студент гр. 3344 _____ Мурдасов М.К.

Преподаватель _____ Иванов Д.В.

Санкт-Петербург

2024

ЗАДАНИЕ
НА КУРСОВУЮ РАБОТУ

Студент: Мурдасов М.К.

Группа: 3344

Тема: Выбор и реализация наиболее подходящей структуры данных.

Исходные данные:
Реализовать структуру данных для хранения информации о температуре окружающего воздуха.

Содержание пояснительной записки: титульный лист, аннотация, введение, выполнение работы, тестирование, заключение, список использованных источников, приложение.

Предполагаемый объем:
Не менее 20 страниц (обязательны разделы «Содержание», «Введение», «Выполнение работы», «Заключение»).

Дата выдачи задания: 06.11.24

Дата сдачи реферата: 15.12.2024

Дата защиты реферата: 17.12.2024

Студент Мурдасов М.К.

Преподаватель Иванов Д.В.

АННОТАЦИЯ

Задача курсовой работы состояла в выборе и реализации наиболее эффективной структуры данных для хранения значений температуры воздуха окружающей среды, собираемой с зондов каждую минуту в течении дня. Данные необходимо получать в отсортированном виде для построения гистограммы.

Методы исследования:

- Проведено сравнение структур данных, потенциально подходящих для решения этой задачи.
- Оценена эффективность каждой структуры.
- По результатам анализа выбрана наиболее эффективная структура данных.

Результаты:

- Определено, что красно-черное дерево обеспечивает быстрое сохранение значений температур и поддерживает их упорядоченность, что необходимо для дальнейшего построения гистограммы без отдельной сортировки данных.
- Реализованы операции вставки и поиска количества значений контрольной температуры.
- Реализовано построение гистограммы, отражающей статистику значений температур в течении дня.
- Реализован графический интерфейс.

Результаты работы демонстрируют, что красно-черное дерево является наилучшим выбором для реализации решения данной задачи.

Программа работает корректно. Была реализована визуализация и графический интерфейс для взаимодействия с программой.

СОДЕРЖАНИЕ

АННОТАЦИЯ 3
ВВЕДЕНИЕ 5
ВЫПОЛНЕНИЕ РАБОТЫ 6
Определение требований 6
Выбор структуры данных 6
Реализация структуры данных 9
Реализация функционала программы 9
Визуализация работы программы 10
ТЕСТИРОВАНИЕ 12
ЗАКЛЮЧЕНИЕ 16
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 17
ПРИЛОЖЕНИЕ 18

ВВЕДЕНИЕ

Цель работы: выбрать наиболее оптимальную и эффективную структуру данных для хранения данных о значениях температуры и дальнейшего составления гистограммы.

Основные задачи:

1. Выбор оптимальной структуры данных для решения данной задачи.
2. Реализация выбранной структуры данных.
3. Демонстрация работы программы.

Методы решения:

1. Анализ требований к структуре.
2. Изучение теоретических основ существующих структур данных.
3. Сравнение структур данных, которые соответствуют поставленным требованиям.
4. Выбор наиболее подходящей структуры с учетом всех факторов.
5. Реализация ключевых операций с выбранной структурой.
6. Проведение тестирования программы для выявления возможных ошибок.
7. Разработка визуализации работы программы с помощью графического интерфейса.

ВЫПОЛНЕНИЕ РАБОТЫ

Определение требований

Проанализировав условие задачи, можно сделать вывод, что структура данных должна обладать следующими свойствами:

1) Эффективная вставка элементов («Зонды собирают информацию о температуре каждую минуту и заносят в базу данных»)
2) Эффективно обрабатывать большое количество данных (ежедневно будет записываться свыше 1000 значений)
3) Упорядоченный вывод. Для построения результирующей гистограммы требуются упорядоченные данные. Структура должна позволять извлекать данные без дополнительной сортировки. Это позволит избежать лишней сложности алгоритма при выводе.

Выбор структуры данных

Основными критериями являются быстрая вставка и упорядоченный вывод. В первую очередь стоит отказаться от выбора таких структур как: стек, очередь, развернутый связный список. Так как устройство данных структур по определению не подходит для решения поставленной задачи. На первый взгляд решением должно являться сбалансированное дерево (например, АВЛ-дерево или красно-черное дерево) ввиду того, что у сбалансированных деревьев достаточно быстрые вставка и поиск, также они позволяют хранить данные в упорядоченном виде. Однако, также стоит взять во внимание хеш-таблицу, которая с большим отрывом выигрывает у деревьев по скорости вставки в лучшем случае и поиска элементов, хотя и не упорядочивает данные.

Для более наглядного сравнения выбранных структур была составлена таблица оценки сложности операций

Таблица 1. Оценка теоретической сложности.

Структура данных	Вставка в лучшем случае	Вставка в худшем случае	Упорядоченный вывод	Необходимость сортировки при выводе
Хеш-таблица	O(1)	O(n)	O(nlog n)	Да (если есть коллизии или неупорядоченные ключи)
АВЛ-дерево	O(log n)	O(log n)	O(n)	Нет (упорядочено по определению)
КЧД-дерево	O(log n)	O(log n)	O(n)	Нет (упорядочено по определению)

Теперь стоит разобрать каждую структуру подробнее.

Хеш-таблица. Согласно оценке теоретической сложности, вставка в лучшем случае занимает O(1). Кажется, что для решения текущей задачи лучше варианта не придумать, но на практике сложность вставки напрямую будет зависеть от качества подобранной хеш-функции, количества коллизий и методе их разрешения (в нашем случае коллизий не избежать, так как диапазон значений температур, скорее всего, будет небольшим). В худшем случае хеш-таблица проигрывает деревьям в скорости вставки. На фактическую сложность вставки влияет слишком много чтобы назвать эту структуру наиболее подходящей для решения поставленной задачи. Также хеш-таблица хранится непрерывным отрезком в памяти и при переполнении программа вынуждена перезаписывать ее с нуля. Ко всему прочему, хеш-таблица не гарантирует упорядоченность данных и при извлечении данных потребуется дополнительная сортировка, что увеличивает сложность извлечения данных в нужном для решения задачи формате.

АВЛ-дерево. В случае АВЛ-дерева вставка (приоритетная для задачи), как и поиск имеют фиксированную сложность O(log n). Это обусловлено постоянными балансировками и сохранением структуры, при которой разница между высотами поддеревьев не превышает 1. В отличии от хеш-таблиц элементы АВЛ-дерева хранятся кусками в памяти, что позволяет существенно выигрывать в потреблении памяти и в сложности. Также в АВЛ-дереве все элементы хранятся упорядоченно, что позволяет извлечь их в необходимом для задачи виде без использования сторонних сортировок, что также упрощает работу.

Взглянув на количество преимуществ АВЛ-дерева в сравнении с хеш-таблицей можно смело исключить хеш-таблицу из сравнения. Далее выбор будет стоять между АВЛ-деревом и красно-черным деревом.

КЧ-дерево (Красно-черное дерево). КЧ-дерево обладает такой же временной сложностью операций, что и АВЛ-дерево. Оно обладает такими же свойствами хранения элементов, но в отличии от АВЛ-дерева, балансировка не является такой строгой и разница между поддеревьями может быть не более чем в 2 раза.

На первый взгляд АВЛ-дерево и КЧ-дерево одинаково хороши для решения задачи. Однако из-за более строгой балансировки АВЛ-деревья быстрее справляются с поиском элементов, а КЧ-деревья со вставкой и удалением, в силу менее строгой балансировки. Это объясняется тем, что АВЛ-дерево, в отличии от КЧ-дерева, выполняет балансировку после каждой вставки и удаления.

Таким образом, так как самая приоритетная операция в данной задаче - вставка, а упорядоченным хранением данных обладает как АВЛ-дерево, так и КЧ-дерево, то для решения задачи было выбрано красно-черное дерево.

Реализация структуры данных

Был реализован класс *Node*, содержащий следующие поля:

— *key* - значение температуры в °C

— *color* - цвет узла

— *left* - левый потомок

— *right* - правый потомок

— *parent* - родитель

— c*ount* - количество повторений температуры

Был реализован класс *RedBlackTree*, содержащий следующие методы:

— *insert(key)* - выполняет вставку элемента.

— *_fix_insert(node)* - восстанавливает свойства красно-черного дерева после вставки. (при необходимости)

— *_left_rotate(node)* - выполняет левый поворот дерева.

— *_right_rotate(node)* - выполняет правый поворот дерева.

— *inorder(self)* - выполняет *inorder* обход и извлекает данные для построения гистограммы.

— *_inorder_helper(node, nodes)* - вспомогательный метод для *inorder()*.

— *find_count(key)* - выполняет поиск значения температуры и возвращает количество повторений.

Реализация функционала программы

Был реализован класс *TemperatureApp*, содержащий следующие методы:

— *add_temperature()* – сохраняет новое значение температуры в программу. Является оберткой над *insert()*.

— *search_temperature()* – выполняет поиск количества повторений контрольной температуры и возвращает его. Является оберткой над *find_count()*.

— *show_histogram()* – извлекает данные о температурах из структуры, делит их на 5 диапазонов и формирует гистограмму.

— *load_temperatures_from_file()* – сохраняет в программу значения температур из файла.

Визуализация работы программы

Для взаимодействия со структурой и визуализации работы программы с помощью *tkinter* был реализован простейший графический интерфейс.

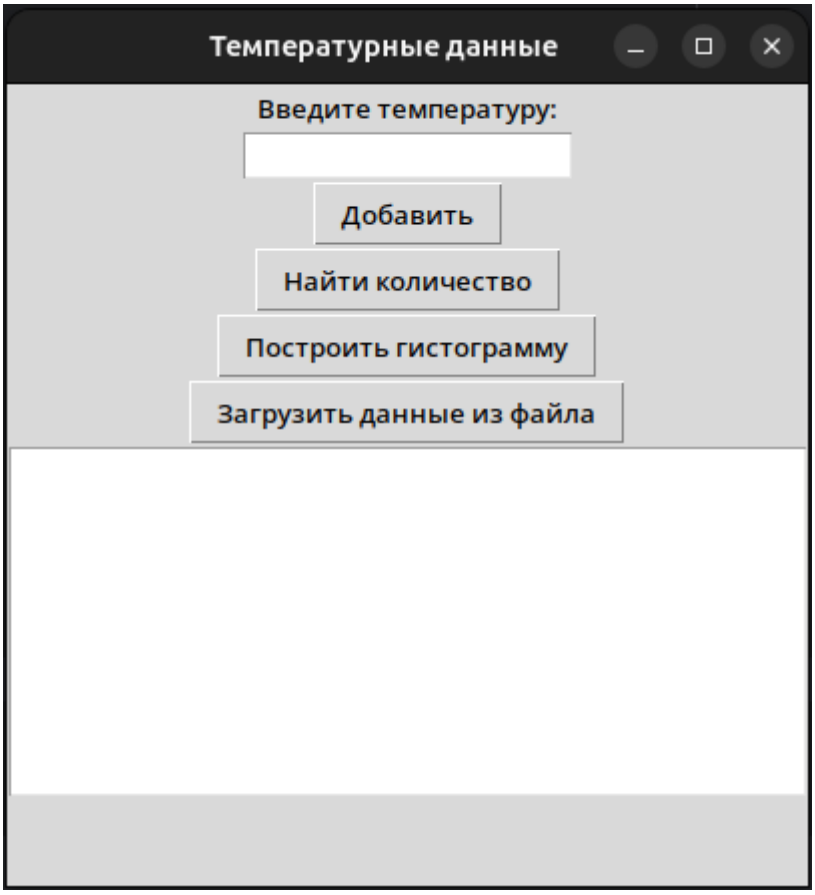

Рисунок 1. Графический интерфейс

Пользователю требуется вносить значения температур посредством записи значения в окно ввода и нажатия кнопки «Добавить». С помощью кнопки «Загрузить данные из файла» пользователь может вызвать диалоговое окно для выбора файла загрузки, после чего данные из файла сохранятся в программе. Для просмотра количества повторений контрольной температура необходимо ввести

ее в окно ввода и нажать «Найти количество». А для отображения гистограммы требуется нажать «Построить гистограмму».

ТЕСТИРОВАНИЕ

Тестирование программы отражено в файле tests.py (см. Приложение). Примеры работы программы приведены на рисунках 1-8.

```
347     cli = TemperatureApp()
348     cli.add_temperature(1)
349     cli.add_temperature(0)
350     cli.add_temperature(-3)
351     cli.add_temperature(1)
352
```

```
cw
/home/chesluchilos/PycharmProjects/cw/.
Добавлена температура: 1
Добавлена температура: 0
Добавлена температура: -3
Добавлена температура: 1
```

Рисунок 2. Добавление значений температуры.

```
347     cli = TemperatureApp()
348     cli.add_temperature(1)
349     cli.add_temperature(0)
350     cli.add_temperature(-3)
351     cli.add_temperature(1)
352     cli.search_temperature(0)
353     cli.search_temperature(1)
```

```
cw
/home/chesluchilos/PycharmProjects/cw/.
Добавлена температура: 1
Добавлена температура: 0
Добавлена температура: -3
Добавлена температура: 1
Температура 0 зафиксирована 1 раз(а).
Температура 1 зафиксирована 2 раз(а).
```

Рисунок 3. Поиск количества повторений температуры.

```
353        cli.search_temperature(1)
354        cli.load_temperatures_from_file("temp.txt")
```

cw

```
Температура 1 зафиксирована 2 раз(а).
Добавлена температура из файла: 12
Добавлена температура из файла: 34
Добавлена температура из файла: 65
Добавлена температура из файла: 13
Добавлена температура из файла: 14
Добавлена температура из файла: 15
Добавлена температура из файла: 16
Добавлена температура из файла: 17
Добавлена температура из файла: 18
Добавлена температура из файла: 20
```

Рисунок 4. Загрузка данных из файла.

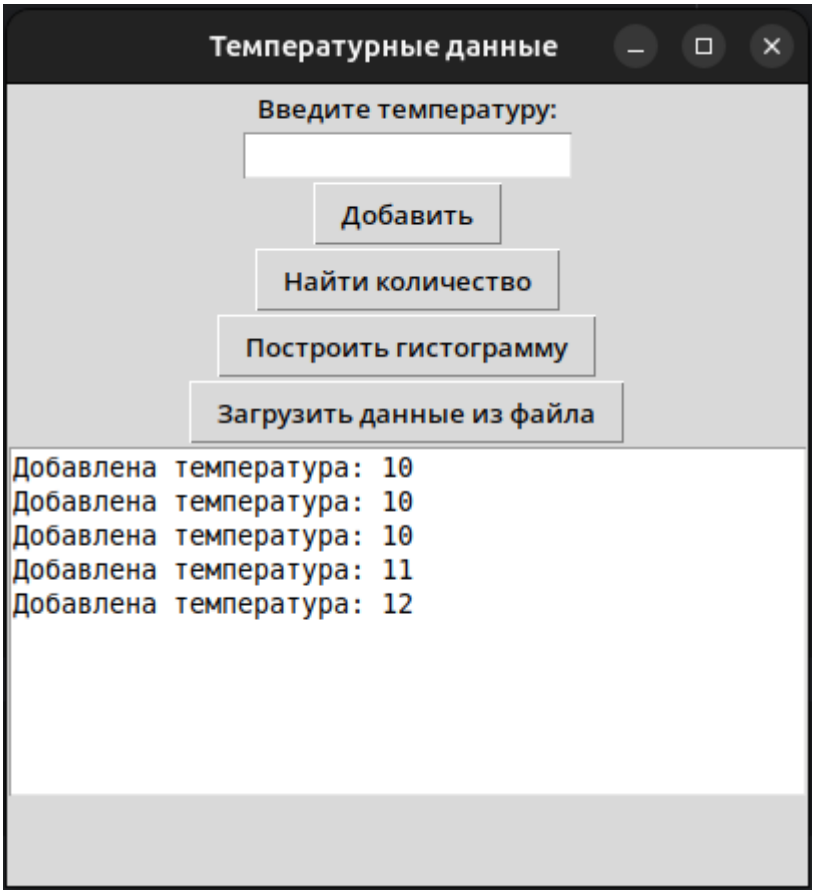

Рисунок 5. Добавление значений температуры. (GUI)

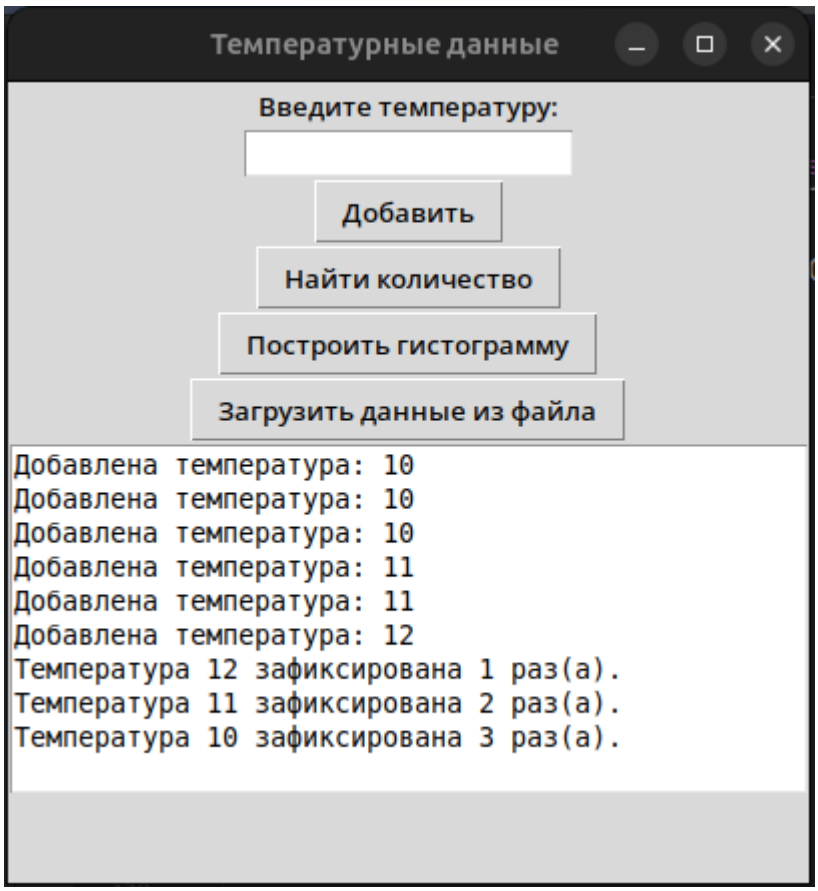

Рисунок 6. Поиск количества повторений температуры. (GUI)

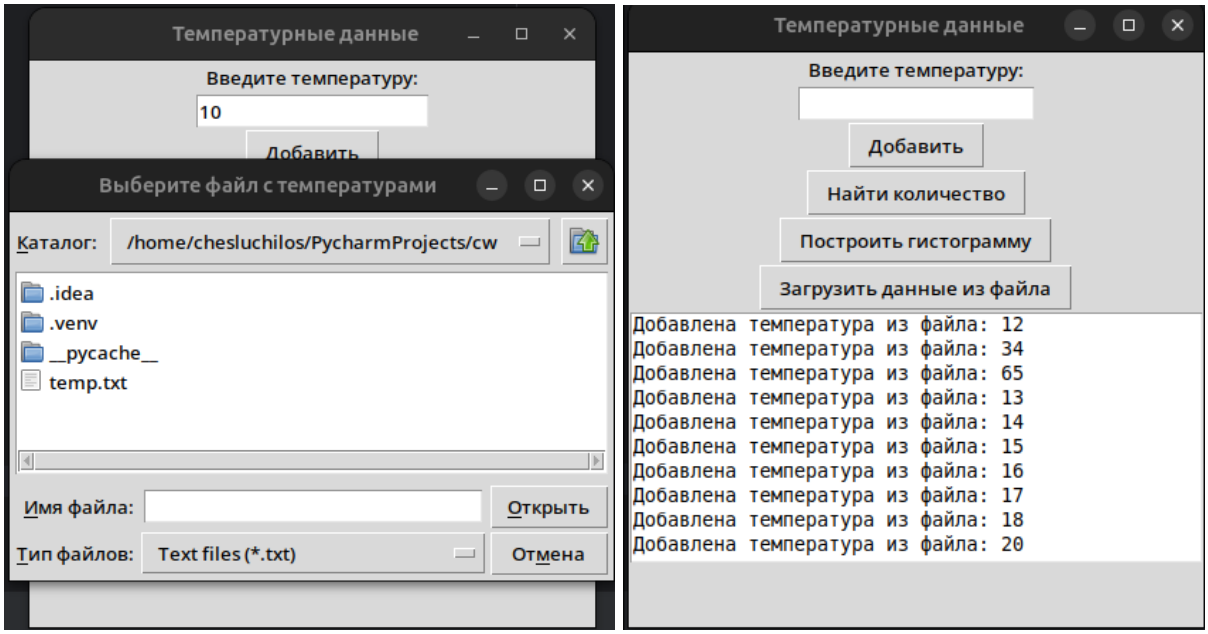

Рисунок 7. Загрузка данных из файла. (GUI)

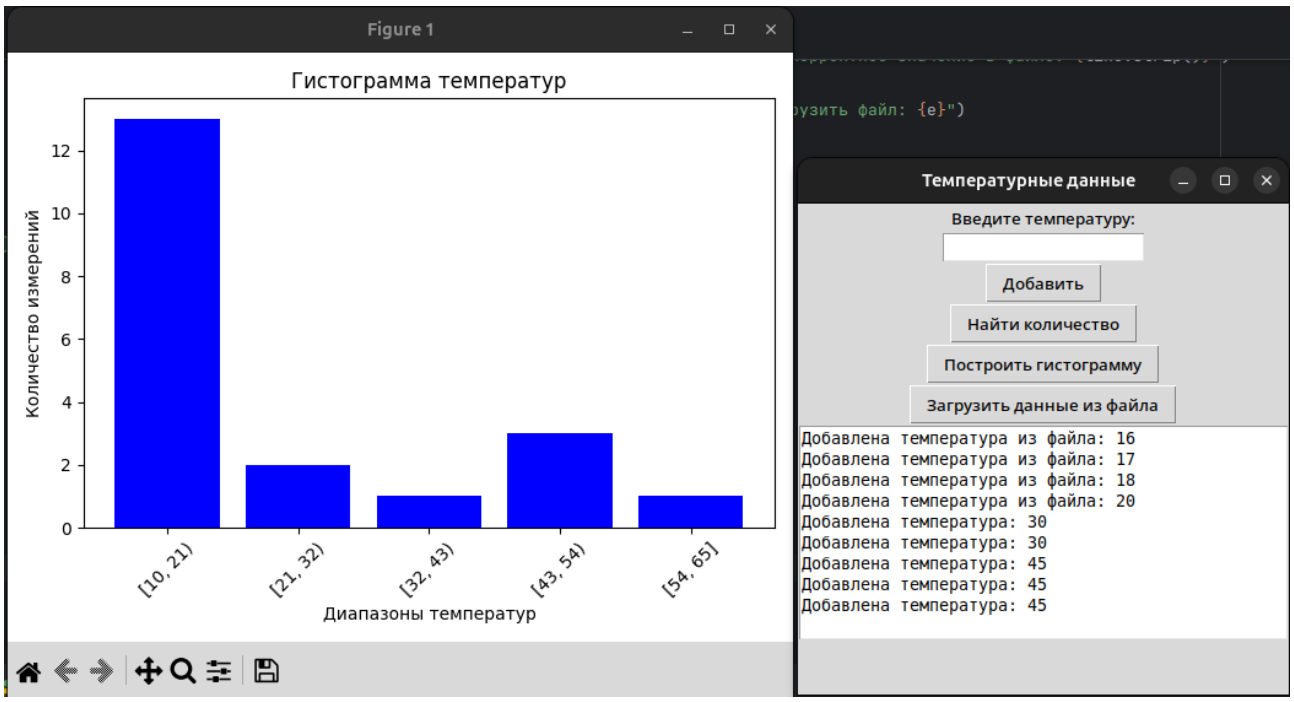

Рисунок 8. Построение гистограммы.

ЗАКЛЮЧЕНИЕ

В результате выполнения курсовой работы был проведен анализ структур данных, включающий в себя анализ свойств каждой структуры данных, сравнение оценок сложности операций каждой структуры и анализ способов хранения данных. По результатам анализа в качестве наиболее подходящей структуры данных была выбрана структура красно-черного дерева. Эта структура способна наиболее эффективно хранить и управлять данными в условиях предложенной задачи.

Была реализована программа по хранению и обработке данных о значениях температуры. Были обработаны возможные ошибки. Также к программе был реализован графический интерфейс.

Программа поддерживает следующие операции:

1. Добавление температуры
2. Поиск количества повторений контрольной температуры
3. Построение гистограммы
4. Загрузка данных о температуре из файла

Таким образом, поставленная задачи выполнены, цель работы достигнута.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Лекционные материалы // se.moevm.info URL: https://se.moevm.info/doku.php/courses:algorithms_structures:start (дата обращения: 12.12.2024)
2. Хеш-таблицы / Хабр // habr.com URL: https://habr.com/ru/articles/509220/ (дата обращения: 12.12.2024)
3. Хеш-таблица – Википедия // ru.wikipedia.org URL: https://ru.wikipedia.org/wiki/Хеш-таблица
4. Красно-чёрное дерево – Википедия // ru.wikipedia.org URL: https://ru.wikipedia.org/wiki/Красно-чёрное_дерево (дата обращения: 12.12.2024)
5. Красно-черные деревья: коротко и ясно / Хабр // habr.com URL: https://habr.com/ru/articles/330644/ (дата обращения: 12.12.2024)
6. АВЛ-дерево – Википедия // ru.wikipedia.org URL: https://ru.wikipedia.org/wiki/АВЛ-дерево (дата обращения: 12.12.2024)
7. АВЛ-деревья / Хабр // habr.com URL: https://habr.com/ru/articles/150732/ (дата обращения: 12.12.2024)
8. Tkinter в Python // selectel.ru URL: https://selectel.ru/blog/tutorials/tkinter-library-in-python/ (дата обращения: 12.12.2024)
9. Red Black Tree vs AVL Tree - GeeksForGeeks // www.geeksforgeeks.org URL: https://www.geeksforgeeks.org/red-black-tree-vs-avl-tree/ (дата обращения: 12.12.2024)

ПРИЛОЖЕНИЕ

ИСХОДНЫЙ КОД ПРОГРАММЫ

Название файла: Node.py

```
class Node:
    def __init__(self, key):
        self.key = key
        self.color = "RED"
        self.left = None
        self.right = None
        self.parent = None
        self.count = 1
```

Название файла: RedBlackTree.py

```
from Node import Node

class RedBlackTree:
    def __init__(self):
        self.TNULL = Node(None)
        self.TNULL.color = "BLACK"
        self.root = self.TNULL
        self.min_key = float('inf')
        self.max_key = float('-inf')

    def insert(self, key):
        new_node = Node(key)
        new_node.left = self.TNULL
        new_node.right = self.TNULL
        parent = None
        current = self.root

        while current != self.TNULL:
            parent = current
            if key == current.key:
                current.count += 1
                return
            elif key < current.key:
                current = current.left
            else:
                current = current.right

        new_node.parent = parent
        if not parent:
            self.root = new_node
        elif key < parent.key:
            parent.left = new_node
        else:
            parent.right = new_node

        if (new_node.parent is None):
            new_node.color = "BLACK"
            return

        if (new_node.parent.parent == None):
            return
```

```
        self.min_key = min(self.min_key, key)
        self.max_key = max(self.max_key, key)

        self._fix_insert(new_node)

    def _fix_insert(self, node):
        while node != self.root and node.parent.color == "RED":
            if node.parent == node.parent.parent.right:
                uncle = node.parent.parent.left
                if uncle.color == "RED":
                    uncle.color = "BLACK"
                    node.parent.color = "BLACK"
                    node.parent.parent.color = "RED"
                    node = node.parent.parent
                else:
                    if node == node.parent.left:
                        node = node.parent
                        self._right_rotate(node)
                    node.parent.color = "BLACK"
                    node.parent.parent.color = "RED"
                    self._left_rotate(node.parent.parent)
            else:
                uncle = node.parent.parent.right
                if uncle.color == "RED":
                    uncle.color = "BLACK"
                    node.parent.color = "BLACK"
                    node.parent.parent.color = "RED"
                    node = node.parent.parent
                else:
                    if node == node.parent.right:
                        node = node.parent
                        self._left_rotate(node)
                    node.parent.color = "BLACK"
                    node.parent.parent.color = "RED"
                    self._right_rotate(node.parent.parent)
        self.root.color = "BLACK"

    def _left_rotate(self, node):
        y = node.right
        node.right = y.left
        if y.left != self.TNULL:
            y.left.parent = node
        y.parent = node.parent
        if not node.parent:
            self.root = y
        elif node == node.parent.left:
            node.parent.left = y
        else:
            node.parent.right = y
        y.left = node
        node.parent = y

    def _right_rotate(self, node):
        y = node.left
        node.left = y.right
        if y.right != self.TNULL:
            y.right.parent = node
```

```
        y.parent = node.parent
        if not node.parent:
            self.root = y
        elif node == node.parent.right:
            node.parent.right = y
        else:
            node.parent.left = y
        y.right = node
        node.parent = y

    def inorder(self):
        nodes = []
        self._inorder_helper(self.root, nodes)
        return nodes

    def _inorder_helper(self, node, nodes):
        if node != self.TNULL:
            self._inorder_helper(node.left, nodes)
            nodes.append((node.key, node.count))
            self._inorder_helper(node.right, nodes)

    def find_count(self, key):
        current = self.root
        while current != self.TNULL:
            if key == current.key:
                return current.count
            elif key < current.key:
                current = current.left
            else:
                current = current.right
        return 0
```

Название файла: TemperatureApp.py

```
import tkinter as tk
from tkinter import messagebox, filedialog
import matplotlib.pyplot as plt
from RedBlackTree import RedBlackTree

class TemperatureApp:
    def __init__(self, root):
        self.tree = RedBlackTree()

        root.title("Температурные данные")
        root.geometry("400x400")

        self.label = tk.Label(root, text="Введите температуру:")
        self.label.pack()

        self.entry = tk.Entry(root)
        self.entry.pack()

        self.add_button = tk.Button(root, text="Добавить",
command=self.add_temperature)
        self.add_button.pack()

        self.search_button = tk.Button(root, text="Найти количество",
command=self.search_temperature)
```

```
        self.search_button.pack()

        self.histogram_button    =    tk.Button(root,    text="Построить
гистограмму", command=self.show_histogram)
        self.histogram_button.pack()

        self.load_button  =  tk.Button(root,  text="Загрузить  данные  из
файла", command=self.load_temperatures_from_file)
        self.load_button.pack()

        self.output = tk.Text(root, height=10, width=50)
        self.output.pack()

    def add_temperature(self):
        try:
            temp = int(self.entry.get())
            self.tree.insert(temp)
            self.output.insert(tk.END,      f"Добавлена      температура:
{temp}\n")
            self.output.see(tk.END)
            print(f"Добавлена температура: {temp}")
        except ValueError:
            messagebox.showerror("Ошибка", "Введите целое число!")

    def search_temperature(self):
        try:
            temp = int(self.entry.get())
            count = self.tree.find_count(temp)
            self.output.insert(tk.END, f"Температура {temp} зафиксирована
{count} раз(а).\n")
            self.output.see(tk.END)
            print(f"Температура {temp} зафиксирована {count} раз(а).")
        except ValueError:
            messagebox.showerror("Ошибка", "Введите целое число!")

    def show_histogram(self):
        nodes = self.tree.inorder()
        if not nodes:
            self.output.insert(tk.END, "Нет данных для гистограммы.\n")
            self.output.see(tk.END)
            return

        temperatures = [node[0] for node in nodes]

        min_temp = min(temperatures)
        max_temp = max(temperatures)

        min_temp = int(min_temp) - 1
        max_temp = int(max_temp) + 1

        num_bins = 5

        step = max((max_temp - min_temp) / num_bins, 1)

        bins = [min_temp + step * i for i in range(num_bins + 1)]
        bins = [round(b) for b in bins]
```

```
        histogram = [0] * num_bins
        for elem in nodes:
            for i in range(num_bins):
                if (bins[i] <= elem[0] < bins[i + 1]) or (i == num_bins -
1 and bins[i] <= elem[0] <= bins[i + 1]):
                    histogram[i] += elem[1]
                    break

        bin_labels = [f"[{bins[i]}, {bins[i + 1]})" for i in range(num_bins
- 1)]
        bin_labels.append(f"[{bins[-2]}, {bins[-1]}]")

        print("Текстовая гистограмма:")
        for i in range(num_bins):
            print(f"{bin_labels[i]}: {'*' * histogram[i]}")

        plt.bar(bin_labels, histogram, color='blue')
        plt.xlabel("Диапазоны температур")
        plt.ylabel("Количество измерений")
        plt.title("Гистограмма температур")
        plt.xticks(rotation=45)
        plt.tight_layout()
        plt.show()

    def load_temperatures_from_file(self):
        file_path  =  filedialog.askopenfilename(title="Выберите  файл  с
температурами",
                                                 filetypes=[("Text  files",
"*.txt")])
        if file_path:
            try:
                with open(file_path, "r") as file:
                    for line in file:
                        try:
                            temp = int(line.strip())
                            self.tree.insert(temp)
                            self.output.insert(tk.END,        f"Добавлена
температура из файла: {temp}\n")
                            self.output.see(tk.END)
                            print(f"Добавлена   температура   из   файла:
{temp}")
                        except ValueError:
                            messagebox.showerror("Ошибка", f"Некорректное
значение в файле: {line.strip()}")
            except Exception as e:
                messagebox.showerror("Ошибка",  f"Не  удалось  загрузить
файл: {e}")
```

Название файла: main.py

```
from TemperatureApp import TemperatureApp
import tkinter as tk

def main():
    root = tk.Tk()
    app = TemperatureApp(root)
    root.mainloop()
```

```
if __name__ == "__main__":
    main()
```

Название файла: tests.py

```
import pytest
from RedBlackTree import RedBlackTree

def test_insert():
    rbt = RedBlackTree()
    rbt.insert(2)
    rbt.insert(0)
    rbt.insert(1)
    assert [node[0] for node in rbt.inorder()] == [0,1,2]
    assert rbt.find_count(0) == 1
    assert rbt.find_count(1) == 1
    assert rbt.find_count(2) == 1

def test_search_repeat():
    rbt = RedBlackTree()
    rbt.insert(2)
    rbt.insert(0)
    rbt.insert(1)
    rbt.insert(1)
    rbt.insert(1)
    assert rbt.find_count(0) == 1
    assert rbt.find_count(-1) == 0
    assert rbt.find_count(1) == 3

def test_inorder():
    rbt = RedBlackTree()
    rbt.insert(2)
    rbt.insert(0)
    rbt.insert(1)
    rbt.insert(1)
    rbt.insert(1)
    assert rbt.inorder() == [(0,1),(1,3),(2,1)]
```